

Support Vector Machines (SVMs)

1. Support Vector Machine (SVM)

A Support Vector Machine (SVM) is a powerful and versatile machine learning model. SVMs are capable of performing linear or nonlinear classification, regression, and even novelty detection. SVMs shine with small to medium-sized nonlinear datasets (i.e., hundreds to thousands of instances), especially for classification tasks.

Limitation: A key drawback is that they don't scale very well to very large datasets.

2. Linear SVM Classification

SVMs can be best understood through visual examples. In the iris dataset, the two classes can be separated with a straight line. Several linear classifiers could be used, but not all perform well. Some classifiers may separate the classes in training but place the boundary too close to the data points, making them unreliable for new data. In contrast, the SVM chooses the boundary that maximizes the distance (**margin**) from the nearest training points on both sides. This approach is known as **large margin classification**.

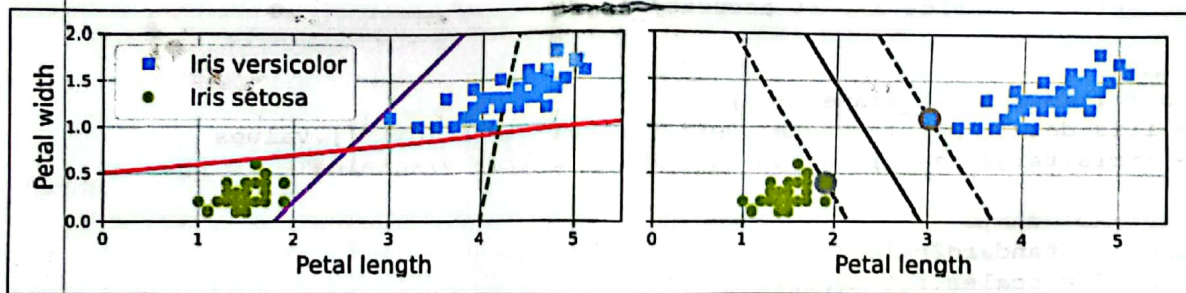


Figure 8-1. Large margin classification
تحدد قيم الفصل

SVMs are highly sensitive to the scale of features. When one feature has a much larger range than another the SVM's decision boundary becomes distorted, resulting in a nearly horizontal margin. After applying feature scaling, all features are brought to similar ranges, and the SVM can find a more appropriate and balanced decision boundary, as shown in the right plot.

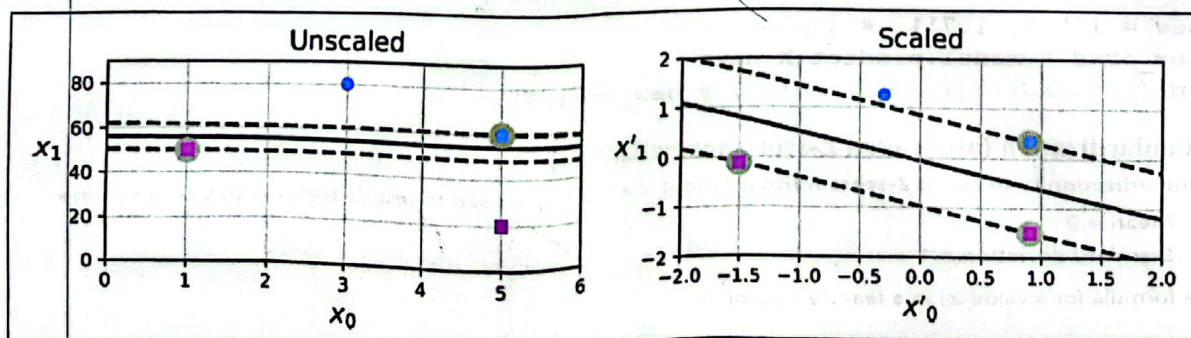


Figure 8-2. Sensitivity to feature scales
الحافة الفاصلة العينة

3. Hard margin classification

which requires every training instance to stay outside the margin and on the correct side of the decision boundary. However, this approach has two major problems: it only works when the data is perfectly linearly separable, and it is extremely sensitive to outliers. Even a single outlier can prevent a hard margin or push the boundary into a poor position, reducing generalization.

4. **Soft margin classification**, which allows some margin violations. The goal is to find a balance between keeping the margin wide and allowing a limited number of misclassified or margin-crossing points.

In **Scikit-Learn**, this balance is controlled by the regularization parameter **C**.

- A low **C** value creates a wider margin but allows more violations, reducing the risk of overfitting but potentially causing underfitting.
- A high **C** value forces fewer violations and fits the training data more closely but increases the risk of overfitting.

In the example, the model with **C = 100** generalizes better than the model with **C = 1** because it finds a more appropriate balance.

The following **Scikit-Learn** code loads the iris dataset and trains a linear SVM classifier to detect *Iris virginica* flowers. The pipeline first scales the features, then uses a **LinearSVC** with **C=1**:

```
from sklearn.datasets import load_iris
from sklearn.svm import LinearSVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# تحميل بيانات Iris
iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # مل الزهرة من نوع Iris virginica?

# توحيد البيانات
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# تدريب SVM إنشاء نموذج
model = LinearSVC(C=1, random_state=42)
model.fit(X_scaled, y)

# التنبؤ على بيانات التدريب
y_pred = model.predict(X_scaled)
print("دقة النموذج على بيانات التدريب:", accuracy_score(y, y_pred))

# زهره جديدة
X_new = [[5.5, 1.7]]
y_new_pred = model.predict(X_new)
print("التنبؤ للزهرة الجديدة:", y_new_pred[0])
```

5. Standardization (also called Z-score normalization)

Standardization (also called Z-score normalization) is a technique used to rescale features so that they have:

- Mean = 0
- Standard deviation = 1

The formula for a value x_i in a feature column is:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Where:

- x_i = original value
- μ = mean of the column
- σ = standard deviation of the column
- z_i = standardized value

This is useful for machine learning algorithms (like SVM, logistic regression, k-NN) because they perform better when features are on the same scale.

Example :

Suppose we have a feature column:

$$X = [2, 4, 6]$$

Step 1: Calculate the mean

$$\mu = \frac{2 + 4 + 6}{3} = 4$$

Step 2: Calculate the standard deviation *الانحراف المعياري*

$$\sigma = \sqrt{\frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{3}} = \sqrt{\frac{4+0+4}{3}} = \sqrt{\frac{8}{3}} \approx 1.632$$

Step 3: Standardize each value

$$z_1 = \frac{2-4}{1.632} \approx -1.225$$

$$z_2 = \frac{4-4}{1.632} = 0$$

$$z_3 = \frac{6-4}{1.632} \approx 1.225$$

Result:

$$Z = [-1.225, 0, 1.225]$$

```
from sklearn.preprocessing import StandardScaler
import numpy as np
```

```
# Original data
```

```
X = np.array([[2], [4], [6]])
```

```
# Create scaler
```

```
scaler = StandardScaler()
```

```
# Fit and transform
```

```
X_scaled = scaler.fit_transform(X)
```

```
print(X_scaled)
```